

Answer any four questions taking two from each group

Group A

Q1 a) Define True/False. If false give the correct answer.

- i. Although Java bytecode differ from platform to platform, JVM is always same in all platform.
- ii. To determine which overloaded version of a method, java checks only the number of arguments.
- iii. In Java, source type need to be larger than destination type for automatic type conversion.
- iv. **\$26S4652\$** is an invalid identifier in Java.
- v. In Java, <<< is a bitwise operator that shift left with zero fill.

5

b) What is the output of the following code segment?

```
01 public class IterationExam {
02     public static void main (String[] args) {
03         int i = 0;
04         First: while (i <= 8) {
05             int sum = 0; i++;
06             Second: for (int j = 1; j <= i;) {
07                 sum = sum + j++;
08                 for (int k = 1; k <= j; k++) {
09                     if (k%2!=0) continue;
10                     sum = sum + k;
11                     if (sum>88) break Second;
12                 }
13                 if (i == 3) continue First; }
14             System.out.print(sum + " ");
15         }
16     }
```

8

Handwritten calculations for the loop output:
i=1: sum=1
i=2: sum=1+2=3
i=3: sum=1+2+3=6
i=4: sum=1+2+3+4=10
i=5: sum=1+2+3+4+5=15
i=6: sum=1+2+3+4+5+6=21
i=7: sum=1+2+3+4+5+6+7=28
i=8: sum=1+2+3+4+5+6+7+8=36

c) Consider the following Java program -

```
01 public class OpExam {
02     public static void main (String[] args) {
03         int a = 135, b = 56, c = 71;
04         a = a + c;
05         System.out.println(a + " " + b + " " + c);
06         a = b + c;
07         System.out.println(a + " " + b + " " + c);
08         a = 13; b = 8;
09         System.out.println(a + " " + b + " = " + (a+b));
10         System.out.println(a + " " + b + " = " + (a+b));
11         byte i = 3; a = i * 2; <<<
12         System.out.println(i + " * 2 = " + a);
13         i = -1; a = i * 24; >>>
14         System.out.println(i + " * 24 = " + a);
15         a = i * 24; System.out.println(i + " * 24 = " + a);
16         a = 75; a = 7; System.out.println("75 * 7 = " + a);
17         boolean d = true, e = false;
18         d = e; System.out.println("d * e = " + d);
19         d = e; System.out.println("d * e = " + d);
20     }
```

12

Fill the ☐ with suitable operator that will output as follows:

126 55 72
126 56 71
13 _ 8 = 13
13 _ 8 = 8
3 _ 2 = 12
-1 _ 24 = -1
-1 _ 24 = 255
75 _ 7 = 5
d _ e = true
d _ e = false

Q2 a) Define True/False. If false give the correct answer.

- i. **super()** must always be the first statement executed inside a subclass constructor.
- ii. Java does not support implementation of multiple *interface* in a single class.
- iii. Private members of a class can only be accessible from its own class and subclass.
- iv. Abstract class can have *abstract* constructor.
- v. Methods that are declared as *final* cannot be overridden.

5

b) Write output of the following java program.

5

```

01 public class ConversionExam {
02     public static void main (String args[]) {
03         byte b; short s; int i = 259; double d = 321.123;
04         b = (byte) i; System.out.println("(1)b="+b+" ##i="+i);
05         i = (int) d; System.out.println("(2)i="+i+" ##d="+d);
06         b = (byte) d; System.out.println("(3)b="+b+" ##d="+d);
07     }
08 }

```

- c) Check the following java program carefully. Do you think there is any compilation error? If there is find the errors, correct it using line number and then write the output. For correcting you cannot insert new statement or delete a statement. If there is no error then write only output.

15

```

01 abstract class A {
02     ✓int i; { System.out.println("Welcome 1"); }
03     A(int a) { i = a;
04         ✓System.out.println("A is 3rd.");
05     }
06     void show() { System.out.println("i : " + i); }
07     ✓static { System.out.println("Static 2"); }
08 }
09 class B extends A {
10     int j; { System.out.println("Welcome 2"); }
11     B(int a, int b) { super(a);
12         ✓System.out.println("B is 2nd.");
13         j = b; super.i = 20;
14     }
15     ✓void show() { System.out.println("j : " + j); }
16     ✓static { System.out.println("Static 3"); }
17 }
18 class C extends B {
19     C() { super(2,1);
20         ✓System.out.println("C is 1st.");
21     }
22     → static void show(int a, int b, int c) {
23         ✓System.out.println("a = "+a+", b = "+b+", c = "+c);
24     }
25     void show(String msg) {
26         System.out.println(msg+" i and j: " + i + " " + super.j);
27     }
28     ✓static { System.out.println("Static 1"); }
29 }
30 class MainClass {
31     public static void main(String args[]) {
32         → C.show(1,2,3);
33         → C c = new C(); ✓c.show();
34         → c.show("ERROR!");
35         A a = c; ✓a.show();
36         B b = c; ✓b.show();
37     }
38 }

```

- a) Check the following java program carefully,

15

```

01 class ExceptionExam {
02     public static void main(String args[]) {
03         System.out.println("Exam Started");
04         int a = args.length;
05         if (a == 0) a = 42 / a;
06         System.out.println("a = " + a);
07         try {
08             if (a == 1) a = a / (a - a);
09             try {
10                 if (a == 2) { int c[] = {1}; c[42] = 99;
11                     System.out.println("c[42] = " + c[42]); }
12                 else if (a == 3)
13                     throw new NullPointerException("Null");
14                 else {
15                     System.out.println("Exam Ended Fast");
16                     return; }
17             } catch (ArrayIndexOutOfBoundsException e) {
18                 System.out.println("Divide by 0");
19             } finally { System.out.println("Un-reachable code"); }
20         } catch (ArithmeticException e) {
21             System.out.println("Array index out-of-bounds"); }
22         catch (Exception e) {
23             System.out.println("No Exception"); }
24         finally { System.out.println("Program Completed"); }
25         System.out.println("Exam Ended");
26     }
27 }

```

and write the output for the following commands:

- 1) java ExceptionExam
 2) java ExceptionExam 1
 3) java ExceptionExam 1 2
 4) java ExceptionExam 1 2 3
 5) java ExceptionExam 1 2 3 4
- b) Check the following java program carefully. Do you think there is any compilation error? If there is find the errors, correct it using line number and then write the output. For correcting you cannot insert new statement or delete a statement. If there is no error then write only output.

5x2
=10

```

01 interface A {
02     int i = 10;
03     int j = 20;
04     private interface B {
05         void s();
06     }
07     void show(String msg);
08 }
09 class C implements A {
10     int k = 30;
11     class D implements B {
12         void s() { System.out.println("i = "+i+" j = "+j+" k = "+k); }
13         public void change() { i = 20; j = 10; }
14     }
15     static class E {
16         static void s() {
17             System.out.println("i = "+i+" j = "+j+" k = "+k); }
18         void s(String msg) { System.out.println(msg); }
19     }
20     void show(String msg) { System.out.println(msg+ " " + k); }
21     String show(String msg) { return msg+ " " + k + " " + i; }
22     public static void main(String args[]) {
23         C c = new C();
24         c.show("Error!");
25         System.out.println(c.show("Hello"));
26         D d = new D();
27         d.s();
28         d.change();
29         E.s("Hello!");
30         E.s();
31     }
32 }
  
```

Group B

Q4 a) Define True/False. If false give the correct answer.

5

- Inner class cannot be public.
- Since constructor does not have any return type, it cannot be overloaded.
- Protected members of a class cannot be accessed from non-subclass within same package.
- Abstract class cannot be inherited.
- All methods in an interface must be static.

b) What are the eight primitive types and their sizes (in bits) in Java?

4

c) Define the class *Student* and *Course* for the following program:

10+6

```

01 public class ObjectToClass {
02     public static void main (String[] args) {
03         Course j = new Course("CSE233", 76.5, 3.75, 3.0);
04         System.out.println("Code:"+j.getCode()+"Mark:"+
05             j.getMark()+"GPA:"+j.getGpa()+"Credit:"+j.getCredit());
06         Course c = new Course("CSE133", 72.5, 3.5, 2.0);
07         Student std = new Student(2012331001,"A","CSE");
08         Course courses[] = new Course[2];
09         courses[0] = j; courses[1] = c; std.setCourses(courses);
10         System.out.println("NAME:"+std.getName()+"REG:"+
11             std.getReg()+"DEPT:"+std.getDept()+"\nAVG Marks:"+
12             std.getAvgMarks()+"CGPA:"+std.getCGPA());
13         std.setName("B"); std.setReg(1999213001); std.setDept("EEE");
14         System.out.println(std.getInfo());
15         std = new Student();
16         System.out.println(std.getInfo());
17     }
18 }
  
```

Output of this program:

Code:CSE233,Mark:76.5,GPA:3.75,Credit:3.0
 NAME:A,REG:2012331001,DEPT:CSE
 AVG Marks:74.5,CGPA:3.65
 Name:B,REG:1999213001,DEPT:EEE
 Name:N/A,REG:0,DEPT:N/A

Q5 a) Define True/False. If false give the correct answer.

- All exception types are subclasses of the *Exception* super class.
- The *finally* block will not execute if no exception is thrown or caught.
- Synchronized* methods can only be accessed by one thread at a time.
- Classes that are declared as *final* can be inherited.
- Interface* cannot be extended.

b) Check the following java program carefully. Do you think there is any compilation error? If there is find the errors, correct it using line number and then write the output. For correcting you cannot insert new statement or delete a statement. If there is no error then write only output.

```

01 public class ThreadExam implements Runnable {
02     public ThreadExam() {
03         Thread t = Thread.currentThread();
04         System.out.println("Current Thread: "+t);
05         t.setName("ExamThread"); t.setPriority(4);
06         System.out.println("Current Thread: "+t);
07         printer();
08     }
09     synchronized public void printer() {
10         Thread t = Thread.currentThread();
11         System.out.println("Printer: "+t);
12     }
13     public void run() {
14         Thread t = Thread.currentThread();
15         System.out.println("Current Thread: "+t);
16         try { Thread.sleep(1000); printer();
17         } catch (InterruptedException e) { }
18         System.out.println("ThreadExam Exiting.");
19     }
20     public static void main (String args[]) {
21         Thread t = new Thread(new ThreadExam(), "MainThread");
22         System.out.println("Current Thread: "+t);
23         t.setPriority(6); t.start();
24         try { t.join();
25         } catch (InterruptedException e) { }
26         System.out.println("ThreadMain Exiting.");
27     }
28 }

```

10

c) Check the following java program carefully. Do you think there is any compilation error? If there is find the errors, correct it using line number and then write the output. For correcting you cannot insert new statement or delete a statement. If there is no error then write only output.

```

1 class ScopeExam { int x;
2     public static void main(String args[]) { int x = 10;
3         System.out.println("x is: " + x);
4         for(int x = 0; x < 3; x++){ int y = -1;
5             System.out.println("x is: " + x + " and y is: " + y);
6             y = 100;
7             System.out.println("x is: " + x + " and y is: " + y);
8         }
9         x = 20; y = 50;
10        System.out.println("x is: " + x + " and y is: " + y);
11        ScopeExam scm = new ScopeExam(); scm.show(); }
12    class AnotherScope {
13        AnotherScope() { int y = 8000; x = 3000;
14            System.out.println("x is: " + x + " and y is: " + y); }
15    }
16    void show() { x = 1000; y = 2000;
17        System.out.println("x is: " + x + " and y is: " + y);
18        new AnotherScope();
19    }
20 }

```

10

Q6 a) Summarize in a table how the class members are accessed under same or different class or subclass or package.

6

b) What is *Deadlock*? Write a code using Java threading where a *Deadlock* condition must occurs.

2+10

c) Imagine that you want to synchronize access to objects of a class that was not designed for multithreaded access. That is, the class does not use *synchronized* methods. Further, this class was not created by you, but by a third party, and you do not have access to the source code. Thus, you can't add *synchronized* to the appropriate methods within the class. How can access to an object of this class be synchronized? Show with an example.

7